

Relaxing the range-for loop customization point finding rules

Abstract

This paper proposes relaxing the rules for finding a customization point for a range-for loop; that is, just because either of a member `begin()` or `end()` is found, that should not disable an ADL customization point. This allows writing ADL `begin()/end()` for types that happen to have an `end()` member function in their class hierarchy. Examples of such types are standard streams, and user wrappers that happen to inherit from standard streams.

The proposal in this paper is that, if `_both_` member `begin()` and `end()` are found, then the members are used in range-for. If just one of them is found, members are not used, but the lookup continues to try and find an ADL `begin()/end()` pair. Note that this paper does **not** propose operating with a mixed member/non-member `begin()/end()` pair.

Rationale

It would seem perfectly reasonable to be able to simply range-for-loop over the data of an input stream. The lack of support for that in the standard might not be such a huge surprise, but what may well be surprising is that we have currently painted ourselves into a corner where such support can't be added. In `[ios::seekdir]`, we specify a `seekdir` enumerator `seekdir::end`, which range-for finds, and subsequently commits to using members for range-for traversal of an `istream`.

Furthermore, programmers might want to wrap `istream`s by deriving from them. We may wish to advice against that, but that doesn't, to me, constitute a sufficient reason to forever turn off range-traversal for such wrapper types. In particular, veneer types are seemingly a perfectly reasonable technique that uses inheritance. See the good old synesis.com.au/resources/articles/cpp/veneers.pdf.

Rumination

So, first of all, are the current rules really wrong? Well, when we put finishing touches on range-for at the very end of C++0x, the current lookup rules were decided on, in Madrid 2011. They were based on [N3257](#). In that paper, Jonathan Wakely makes a somewhat convincing case for preferring an ill-formed program if a user provides just a `begin()` but no `end()`, and there happens to be a namespace-scope template that will then be used. That rationale was certainly enough to convince the committee at that time.

However, that rationale relies on the assumption that the user somehow intended to opt in to range-for traversal, failed to do so correctly, and is now surprised that some namespace-scope `begin()/end()` pair was chosen. The problem here is that if the user instead intended to write such a namespace-scope `begin()/end()` pair, and can't change the presence of an 'end' in his class hierarchy, that's just impossible to do.

While it was a nice idea to avoid funny results in the examples depicted in [N3257](#), I think it's an unfortunate consequence that we now prevent perfectly valid use cases written by programmers who know what they are

doing. We shouldn't base our rules on overt worrying about things that could go wrong, but rather allow useful programming techniques to be applied, and rely on tools to hold the hand of a programmer. That is, we used to have a principle of trusting the programmer rather than molly-coddling him.

Is this a significant problem? Probably not, not something that a whole lot of programmers run into. However, the problem seems easy to fix, with not much downside. We have more cases where customization point lookup rules are draconian and prevent reasonable code from working; I have written another paper that deals with a customization point lookup problem with structured bindings. These problems can be worked around with adapter types, but I'd rather have less draconian rules.

An example of code that doesn't work

```
#include <sstream>
#include <iterator>

struct X : std::stringstream
{
    // some other stuff here
};

std::istream_iterator<char> begin(X& x)
{
    return std::istream_iterator<char>(x);
}

std::istream_iterator<char> end(X& x)
{
    return std::istream_iterator<char>();
}

int main()
{
    X x;
    for (auto&& i : x) {
        // do your magic here
    }
}
```

I find it very unreasonable that that code is ill-formed. N3257 suggests using an adapter wrapper as a work-around, so that X would be wrapped into another class which does not inherit from stringstream, and can then cleanly provide the ADL begin()/end() customization points. While that's doable, it's also a fairly heavy-handed approach, especially if this X-wrapper has no other purpose than providing range access. Furthermore, the X-wrapper isn't substitutable for X, since the whole idea of public inheritance of stringstream in X is that there's a standard conversion from X to stringstream; there is none from X-wrapper to stringstream.

Wording

In [stmt.ranged]/1, bullet 1.3.2, edit as follows:

if the for-range-initializer is an expression of class type C, the unqualified-ids begin and end are looked up in the scope of C as if by class member access lookup (6.4.5), and if ~~either~~ ~~(or both)~~ finds at least one declaration, begin-expr and end-expr are __range.begin() and __range.end(), respectively;